

REVISIÓN DE CÓDIGO

Informe de Revisión - Proyecto "gestor-pedidos"

Materia: Programación III / IV - Fundamentos de Spring Boot

Estudiante: Ignacio Salazar (ignaciosalazar986@gmail.com)

Fecha de Revisión: 10 de Agosto de 2026

1. Resumen de Evaluación General

🌟 Aspectos Destacados

- **Estructura de Paquetes:** Excelente división modular en `categoria`, `detallePedido`, `pedido`, `producto` y `usuario` siguiendo fielmente el diagrama de paquetes de la consigna.
- **Uso de Java Records:** Decisión moderna y limpia al implementar DTOs como `record` en Java (ej. `CategoriaCreate`, `CategoriaDTO`, `CategoriaEdit`).
- **Configuración del Proyecto:** Dependencias correctas en `build.gradle` (Spring Data JPA, Spring Web, Lombok, DevTools, H2) y configuración de base de datos en memoria lista en `application.properties`.

2. Hallazgos y Puntos a Corregir

1. Atributo `static` en Entidad (Crítico)

Archivo: `src/main/java/com/gestor_pedidos/entities/Pedido.java` (Línea 27)

Problema: Se declaró `private static Set<DetallePedido> detallePedido = new HashSet<>();`.

Causa/Efecto: Al usar `static`, la colección pertenece a la clase y no a cada pedido individual. Todos los objetos `Pedido` compartirán exactamente los mismos detalles. Además, JPA no permite mapear colecciones estáticas como relaciones.

Solución: Remover el modificador `static`.

2. Condición Lógica Invertida (Bug de Ejecución)

Archivo: `src/main/java/com/gestor_pedidos/producto/ProductoDTO.java` (Línea 8)

Código actual:

```
prod.getCategoria() == null ? CategoriaDTO.toDTO(prod.getCategoria()) : null
```

Causa/Efecto: Si la categoría es `null`, intenta ejecutar `toDTO(null)` provocando un `NullPointerException`. Si no es `null`, devuelve `null`.

Solución: Cambiar `==` por `!=`:

```
prod.getCategoria() != null ? CategoriaDTO.toDTO(prod.getCategoria()) : null
```

3. Métodos de Conversión no Estáticos

Archivos: `PedidoDTO.java` (Línea 8) y `UsuarioDTO.java` (Línea 7)

Problema: Los métodos `toDTO(...)` están definidos como métodos de instancia en lugar de `public static`.

Solución: Agregar la palabra clave `static` para permitir su invocación directa (ej. `PedidoDTO.toDTO(pedido)`).

4. Vinculación Bidireccional de Pedido

Archivos: `Pedido.java` y `DetallePedido.java`

Problema: En `Pedido.addDetallePedido(...)` se crea el `DetallePedido` pero no se le asigna la referencia al pedido actual (`setPedido(this)`).

Solución: Asignar `nuevoDetalle.setPedido(this);` para mantener la consistencia en memoria.

5. Inconsistencia de Nombres de Atributos

Archivos: `Usuario.java` (`contasenia`) vs `UsuarioCreate.java` / `UsuarioDTO.java` (`contrasenia`)

Solución: Unificar el nombre del campo a `contrasenia` en todas las clases.

3. Recomendaciones para Completar la Consigna

Implementación de Inyección de Dependencias y Estereotipos

La consigna solicita demostrar los conceptos de **Inyección de Dependencias por Constructor** y uso de **Estereotipos de Spring** (`@Component`, `@Service`, `@Repository`):

- En lugar de instanciar los DTOs directamente en el método `main` de `GestorPedidosApplication.java`, podés crear un componente inicializador anotado con `@Component` (o implementar `CommandLineRunner`).
- Inyectar dependencias necesarias vía constructor.
- Instanciar los objetos requeridos por la consigna:
 - 2 Usuarios
 - 3 Categorías
 - 10 Productos
 - 3 Pedidos (con al menos 2 detalles cada uno)

Generado automáticamente por Antigravity AI - Asistente de Desarrollo de Software